

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

ING. JORGE ARUN ZAMBRANO CEDEÑO, MG

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: JEAN CARLO CRUZ FAJARDO

PARALELO: 5/1 TECNOLOGIA DE LA INFORMACION

SEMESTRE: SEMESTRE QUINTO

ACTIVIDAD: Actividad # 1: Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Objetivo de la tarea:

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Disponibilidad: 20/04/2026 hasta 16/05/2026

Fecha límite: 16/05/2026

Descripción de la Actividad

El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Calificación Tendrá una ponderación de 10 puntos.

Instrucciones Análisis del problema

- Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas).
- Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).

Diseño del sistema - Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

- Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.



- Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.
- Diseñar la arquitectura general del sistema integrando el flujo de versiones.

Aplicación de patrones de diseño

Utilizar al menos tres patrones. Algunos recomendados:

- Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos.
- Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).
- Factory Method: Para la creación de diferentes tipos de incidentes según el área (Red, Hardware, Software).
- MVC (Model-View-Controller): Como patrón estructural para organizar la lógica de negocio y la interfaz.

Documentación del diseño

- Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.
- Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.
- Incluir diagramas de clases y secuencia donde se refleje la implementación de los patrones.

Recomendaciones

Presentar la tarea con un adecuado formato y presentación de régimen universitario en archivo pdf. Utilizar el formato "Formato_Entrega.docx", estará en la sección de Materiales de aprendizaje / Recursos - Unidad 1 / Recursos

Complementarios de la Unidad 1.

La actividad deberá ser desarrollada en el tiempo estipulado.



La actividad es individual y por lo tanto cada estudiante asumirá la responsabilidad de su autoría. Asegúrate de nombrar y justificar claramente cada patrón. Revisa que los diagramas sean legibles y coherentes con el diseño planteado. Usa nombres significativos para las clases y métodos. No olvides cuidar la presentación del documento: ortografía, redacción clara, numeración de secciones. Puedes usar herramientas como Draw.io, Lucidchart o StarUML para los diagramas.

1. INTRODUCCIÓN

El crecimiento de las infraestructuras tecnológicas en los centros de datos ha incrementado la necesidad de contar con sistemas capaces de gestionar incidencias técnicas de manera organizada, eficiente y escalable. En este contexto, los sistemas Help Desk representan una solución fundamental para el registro, seguimiento y resolución de problemas relacionados con hardware, software y redes, permitiendo mantener la continuidad operativa de los servicios tecnológicos.

Un sistema de gestión de incidentes facilita la comunicación entre usuarios finales, técnicos de soporte y administradores de sistemas, optimizando los tiempos de respuesta y mejorando el control de las incidencias reportadas. Además, la automatización de procesos como la asignación de técnicos, clasificación de prioridades y envío de notificaciones contribuye a incrementar la productividad y la calidad del soporte técnico.

El presente proyecto tiene como objetivo diseñar la arquitectura lógica de un sistema Help Desk orientado a la gestión de incidencias en un Data Center, aplicando principios de ingeniería de software, diagramas UML y patrones de diseño. Para ello, se implementaron patrones como Singleton, Factory Method y Observer, los cuales permiten desarrollar una solución más mantenible, reutilizable y escalable.

Asimismo, se utilizó el modelo de control de versiones Gitflow mediante un repositorio en GitHub, con la finalidad de gestionar de forma organizada el desarrollo del proyecto, manteniendo un flujo de trabajo profesional basado en ramas, integración y control de cambios.

2. ANÁLISIS DEL PROBLEMA

En los entornos tecnológicos modernos, los centros de datos (Data Center) requieren sistemas eficientes para la gestión y resolución de incidencias técnicas relacionadas con



infraestructura de hardware, software y redes. Muchas organizaciones aún manejan estos procesos de forma manual o mediante herramientas limitadas, provocando retrasos en la atención de fallos, pérdida de información, dificultades de seguimiento y una baja trazabilidad de los incidentes reportados.

La ausencia de un sistema centralizado de gestión de incidencias dificulta la coordinación entre los diferentes actores involucrados en el soporte técnico, generando problemas en la asignación de responsabilidades, monitoreo del estado de los tickets y control de tiempos de respuesta. Además, la falta de automatización en las notificaciones y categorización de incidentes puede afectar la disponibilidad de servicios críticos dentro del Data Center.

Para solucionar esta problemática, se propone el diseño de un sistema Help Desk orientado a la gestión de incidentes técnicos, capaz de registrar, clasificar y administrar incidencias de manera organizada y escalable. El sistema permitirá mejorar el flujo de trabajo entre usuarios finales, técnicos de soporte y administradores, optimizando el proceso de atención y resolución de problemas. El proyecto contempla el uso de diagramas UML, principios SOLID, patrones de diseño de software y control de versiones mediante Gitflow en GitHub, con el objetivo de construir una arquitectura lógica profesional, mantenible y adaptable a futuras ampliaciones.

2.1 Actores Principales

Usuario Final: Es el encargado de reportar incidentes relacionados con fallos de hardware, software o red dentro de la infraestructura tecnológica. Puede crear tickets, consultar estados, adjuntar evidencias y dar seguimiento a sus solicitudes.

Técnico de Soporte: Responsable de analizar, diagnosticar y resolver los incidentes asignados. Tiene acceso a funcionalidades relacionadas con actualización de estados, comentarios técnicos y cierre de tickets.

Administrador de Sistemas: Administra la configuración general del sistema, la gestión de usuarios, roles, categorías y supervisión de incidentes. Además, puede generar reportes e indicadores para el monitoreo del soporte técnico.

2.2 Problemas Identificados



- Gestión manual de incidencias técnicas.
- Retrasos en la asignación de técnicos responsables.
- Falta de seguimiento y control de tickets.
- Ausencia de notificaciones automáticas.
- Dificultad para categorizar incidentes.
- Escasa trazabilidad de cambios y versiones del sistema.
- Baja mantenibilidad de sistemas tradicionales.

2.3 Solución Propuesta

La solución propuesta consiste en desarrollar un sistema de gestión de incidentes Help Desk basado en una arquitectura orientada a objetos y principios de ingeniería de software. El sistema permitirá automatizar el registro, clasificación y seguimiento de incidencias técnicas, integrando patrones de diseño como Singleton, Factory Method y Observer para mejorar la organización y escalabilidad de la solución.

Asimismo, se implementará un flujo de trabajo Gitflow para el control de versiones del proyecto, permitiendo mantener un desarrollo estructurado mediante ramas principales, ramas de desarrollo y funcionalidades específicas. Esto facilitará la trazabilidad y el mantenimiento del sistema durante futuras actualizaciones.

3. REQUERIMIENTOS DEL SISTEMA

Los requerimientos del sistema describen las funcionalidades y características necesarias para el correcto funcionamiento del sistema Help Desk orientado a la gestión de incidentes técnicos dentro de un Data Center. Estos requerimientos se clasifican en funcionales y no funcionales.

3.1 Requerimientos Funcionales

Los requerimientos funcionales definen las operaciones y servicios que el sistema debe proporcionar a los usuarios.

Código	Requerimiento Funcional	Descripción
RF1	Registro de usuarios	El sistema debe permitir registrar usuarios según su rol (Usuario Final, Técnico o Administrador).



RF2	Inicio de sesión	El sistema debe permitir la autenticación de usuarios mediante credenciales.
RF3	Creación de tickets	El usuario final debe poder registrar incidentes técnicos mediante tickets.
RF4	Clasificación de incidentes	El sistema debe permitir categorizar incidentes en Hardware, Red o Software.
RF5	Asignación de técnicos	El administrador o sistema debe asignar técnicos especializados según el tipo de incidente.
RF6	Actualización de estados	El técnico debe poder cambiar el estado del incidente (Pendiente, En Proceso, Resuelto, Cerrado).
RF7	Notificaciones automáticas	El sistema debe enviar notificaciones automáticas ante nuevos incidentes o cambios de estado.
RF8	Consulta de tickets	Los usuarios deben poder consultar el historial y estado de sus incidentes.
RF9	Gestión de usuarios	El administrador debe poder crear, modificar o eliminar usuarios del sistema.
RF10	Generación de reportes	El sistema debe permitir generar reportes de incidencias y rendimiento técnico.
RF11	Cierre de tickets	El técnico o administrador debe poder cerrar tickets resueltos.
RF12	Reapertura de tickets	El usuario podrá reabrir tickets en caso de que el problema persista.

3.2 Requerimientos No Funcionales

Los requerimientos no funcionales definen las características de calidad y restricciones técnicas del sistema.

Código	Requerimiento No Funcional	Descripción
RNF1	Escalabilidad	El sistema debe permitir agregar nuevas funcionalidades sin afectar la arquitectura existente.
RNF2	Mantenibilidad	El sistema debe diseñarse bajo principios SOLID y patrones de diseño para facilitar su mantenimiento.
RNF3	Disponibilidad	El sistema debe garantizar disponibilidad continua para la gestión de incidentes críticos.



RNF4	Seguridad	El sistema debe proteger la información mediante autenticación y control de acceso por roles.
RNF5	Rendimiento	El sistema debe responder de forma eficiente ante múltiples solicitudes simultáneas.
RNF6	Trazabilidad	El sistema debe mantener historial de cambios y versiones mediante Gitflow.
RNF7	Usabilidad	La interfaz debe ser intuitiva y fácil de utilizar para todos los actores del sistema.
RNF8	Compatibilidad	El sistema debe poder ejecutarse en diferentes navegadores y entornos tecnológicos.
RNF9	Reutilización	La arquitectura debe permitir reutilizar componentes y módulos del sistema.
RNF10	Integridad de datos	El sistema debe garantizar consistencia y confiabilidad de la información almacenada.

3.3 Restricciones del Sistema

Código	Restricción
RS1	El proyecto debe utilizar UML para representar la arquitectura lógica del sistema.
RS2	El desarrollo debe gestionarse mediante Gitflow en un repositorio de GitHub .
RS3	El sistema debe implementar al menos tres patrones de diseño de software.
RS4	La arquitectura debe seguir principios de programación orientada a objetos y SOLID.
RS5	Los diagramas deben elaborarse utilizando herramientas UML como Draw.io o Lucidchart .

4. DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso permite representar gráficamente las interacciones entre los actores del sistema y las funcionalidades principales del sistema Help Desk. Este modelo facilita la comprensión de los procesos involucrados en la gestión de incidencias técnicas dentro del Data Center.

El sistema contempla tres actores principales: el Usuario Final, encargado de reportar incidentes y consultar el estado de sus tickets; el Técnico de Soporte, responsable de atender y resolver las incidencias; y el Administrador de Sistemas, quien supervisa el funcionamiento general del sistema y administra usuarios y reportes.

4.1 Actores del Sistema



Actor	Descripción
Usuario Final	Reporta incidentes y realiza seguimiento a los tickets generados.
Técnico de Soporte	Atiende, diagnostica y resuelve incidentes técnicos.
Administrador de Sistemas	Gestiona usuarios, supervisa incidentes y genera reportes.

4.2 Casos de Uso Principales

Código	Caso de Uso	Actor Principal
CU1	Iniciar sesión	Todos los actores
CU2	Crear ticket	Usuario Final
CU3	Consultar estado de ticket	Usuario Final
CU4	Clasificar incidente	Sistema / Administrador
CU5	Asignar técnico	Administrador
CU6	Actualizar estado del incidente	Técnico
CU7	Resolver incidente	Técnico
CU8	Cerrar ticket	Técnico / Administrador
CU9	Reabrir ticket	Usuario Final
CU10	Enviar notificaciones	Sistema
CU11	Gestionar usuarios	Administrador
CU12	Generar reportes	Administrador

4.4 Relaciones del Diagrama

El diagrama de casos de uso incluye relaciones de tipo:

Relación	Descripción
Association	Relación directa entre actor y funcionalidad.
Include	Un caso de uso requiere obligatoriamente otro proceso.
Extend	Un caso de uso amplía opcionalmente otro comportamiento.

4.5 Herramienta Utilizada

El diagrama de casos de uso fue elaborado utilizando la herramienta [Draw.io](https://draw.io), permitiendo representar gráficamente las interacciones y procesos principales del sistema Help Desk.

Diagrama de Casos





Enlace de Diagrama de Casos :
<https://drive.google.com/file/d/1L8IzDR6oU0jTpS2635MA4PgRzR5Bo5Tf/view?usp=sharing>

5. EXPLICACIÓN DE PATRONES DE DISEÑO

Los patrones de diseño son soluciones reutilizables aplicadas a problemas comunes dentro del desarrollo de software orientado a objetos. En el presente sistema Help Desk se implementaron los patrones Singleton, Factory Method, Observer y MVC, con el objetivo de mejorar la organización, mantenibilidad, reutilización y escalabilidad de la arquitectura del sistema.

5.1 Patrón Singleton

El patrón Singleton garantiza que una clase tenga una única instancia durante toda la ejecución del sistema y proporciona un punto global de acceso a dicha instancia.

En el sistema Help Desk, este patrón fue implementado en la clase GestorTickets, encargada de administrar los tickets e incidentes del sistema.

Problema que resuelve: El sistema requiere centralizar la gestión de tickets para evitar inconsistencias en:

- ✓ asignación de incidentes
- ✓ actualización de estados
- ✓ control de registros
- ✓ administración de procesos

Si existieran múltiples instancias del gestor de tickets, podrían producirse conflictos y duplicidad de información

Razón de uso: Se utilizó el patrón Singleton para garantizar que exista una única instancia encargada de administrar los tickets del sistema, permitiendo mantener consistencia y control centralizado sobre la información.

Beneficios

- ✓ Evita múltiples instancias innecesarias.
- ✓ Centraliza la gestión de incidentes.
- ✓ Mejora el control del sistema.



- ✓ Facilita el acceso global al gestor de tickets.

5.2 Patrón Factory Method

El patrón Factory Method permite crear objetos sin especificar directamente la clase concreta que será instanciada. En el sistema Help Desk, este patrón fue implementado mediante la clase abstracta IncidenteFactory y sus fábricas concretas:

- ✓ HardwareFactory
- ✓ NetworkFactory
- ✓ SoftwareFactory

Problema que resuelve: El sistema necesita crear diferentes tipos de incidentes dependiendo del área afectada, como:

- ✓ incidentes de hardware
- ✓ incidentes de red
- ✓ incidentes de software

Sin este patrón, la creación de objetos estaría fuertemente acoplada al código principal del sistema.

Razón de uso: Se utilizó Factory Method para permitir la creación flexible y organizada de distintos tipos de incidentes sin modificar la lógica principal del sistema.

Beneficios

- ✓ Reduce el acoplamiento entre clases.
- ✓ Facilita la escalabilidad del sistema.
- ✓ Permite agregar nuevos tipos de incidentes fácilmente.
- ✓ Mejora la reutilización de código.

5.3 Patrón Observer

El patrón Observer establece una relación uno a muchos entre objetos, permitiendo que cuando un objeto cambie de estado, todos sus observadores sean notificados automáticamente. En el sistema Help Desk, este patrón fue utilizado en el sistema de notificaciones automáticas. **Problema que resuelve:** El sistema requiere informar automáticamente a los técnicos y usuarios cuando:



- ✓ se crea un incidente
- ✓ cambia el estado de un ticket
- ✓ se asigna un técnico
- ✓ se cierra un incidente

Sin este patrón, el sistema tendría una fuerte dependencia entre módulos de gestión y notificaciones.

Razón de uso: Se utilizó Observer para implementar un mecanismo de notificaciones desacoplado y reactivo, permitiendo mantener comunicación automática entre componentes del sistema.

Beneficios

- ✓ Permite notificaciones automáticas.
- ✓ Reduce el acoplamiento entre módulos.
- ✓ Facilita mantenimiento y extensibilidad.
- ✓ Mejora la programación reactiva del sistema.

5.4 Patrón MVC (Model – View – Controller)

El patrón MVC separa la aplicación en tres componentes principales:

Componente	Función
Model	Gestiona datos y lógica de negocio
View	Representa la interfaz visual
Controller	Gestiona las acciones y flujo de la aplicación

En el sistema Help Desk, el patrón MVC organiza el sistema de forma estructurada y modular.

Problema que resuelve: En sistemas complejos, mezclar interfaz gráfica, lógica de negocio y manejo de datos provoca:

- ✓ dificultad de mantenimiento
- ✓ bajo nivel de reutilización
- ✓ código desorganizado



Razón de uso: Se utilizó MVC para separar responsabilidades dentro del sistema y mantener una arquitectura clara y profesional.

Beneficios

- ✓ Mejora la organización del código.
- ✓ Facilita mantenimiento y escalabilidad.
- ✓ Permite reutilización de componentes.
- ✓ Separa claramente responsabilidades.
- ✓ Mejora el desarrollo modular.

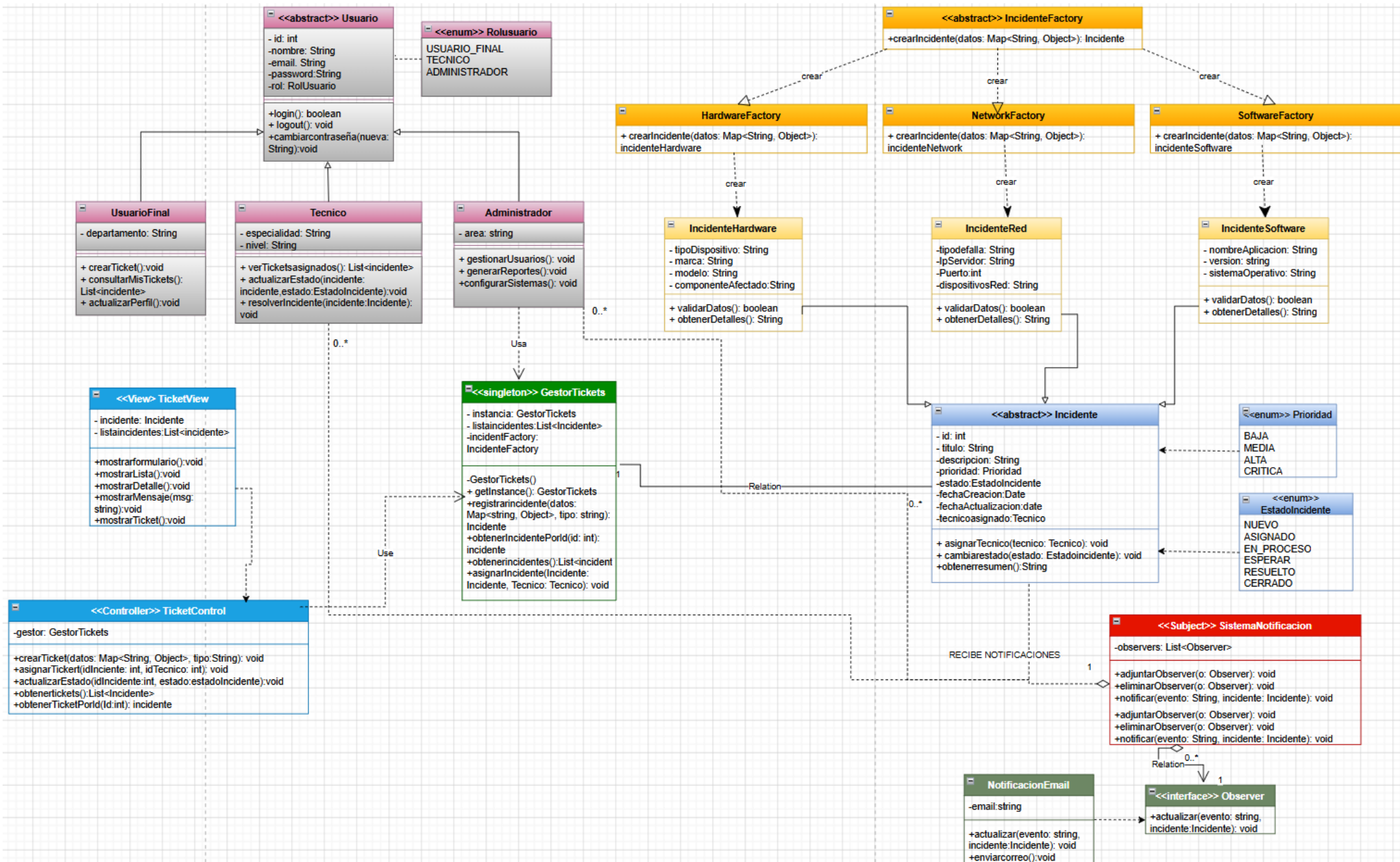
5.5 Integración de los Patrones en el Sistema

La combinación de los patrones Singleton, Factory Method, Observer y MVC permitió construir una arquitectura lógica robusta y mantenible para el sistema Help Desk. Cada patrón cumple una función específica dentro del sistema:

Patrón	Función Principal
Singleton	Centralizar la gestión de tickets
Factory Method	Crear distintos tipos de incidentes
Observer	Gestionar notificaciones automáticas
MVC	Organizar la arquitectura del sistema

La aplicación conjunta de estos patrones mejora la calidad del software y facilita futuras ampliaciones del sistema.

Diagrama de UML





Enlace de Diagrama de UML:
<https://drive.google.com/file/d/1L8IzDR6oU0jTpS2635MA4PgRzR5Bo5Tf/view?usp=sharing>

6. JUSTIFICACIÓN DE LOS PRINCIPIOS SOLID

Los principios SOLID representan un conjunto de buenas prácticas de diseño orientado a objetos que permiten desarrollar sistemas más mantenibles, escalables y reutilizables. En el sistema Help Desk propuesto, estos principios fueron considerados durante el diseño de la arquitectura lógica y la implementación de los patrones de diseño. La aplicación de SOLID contribuye a reducir el acoplamiento entre componentes, mejorar la organización del código y facilitar futuras ampliaciones del sistema sin afectar su funcionamiento.

6.1 Single Responsibility Principle (SRP)

Principio: Este principio establece que una clase debe tener una única responsabilidad o motivo de cambio.

Aplicación en el sistema: En el sistema Help Desk, cada clase fue diseñada para cumplir una función específica.

Clase	Responsabilidad
GestorTickets	Administrar tickets e incidentes
SistemaNotificacion	Gestionar notificaciones
TicketView	Mostrar información al usuario
TicketController	Controlar el flujo de operaciones
IncidenteFactory	Crear objetos de incidentes

Beneficio

- ✓ Facilita mantenimiento.
- ✓ Reduce complejidad.
- ✓ Mejora organización del sistema.

6.2 Open/Closed Principle (OCP)



Principio: Las clases deben estar abiertas para extensión, pero cerradas para modificación.

Aplicación en el sistema: El patrón Factory Method permite agregar nuevos tipos de incidentes sin modificar el código existente. Actualmente existen:

- ✓ IncidenteHardware
- ✓ IncidenteRed
- ✓ IncidenteSoftware

En el futuro se podría agregar: IncidenteSeguridad / sin alterar las clases ya implementadas.

Beneficio

- ✓ Facilita escalabilidad.
- ✓ Reduce riesgos de errores.
- ✓ Permite crecimiento modular del sistema.

6.3 Liskov Substitution Principle (LSP)

Principio: Las clases hijas deben poder sustituir a sus clases padre sin alterar el funcionamiento del sistema.

Aplicación en el sistema

Las clases:

- ✓ IncidenteHardware
- ✓ IncidenteRed
- ✓ IncidenteSoftware

pueden utilizarse como objetos de tipo Incidente, respetando la herencia y comportamiento esperado.

Beneficio

- ✓ Mejora reutilización.
- ✓ Garantiza consistencia en la herencia.
- ✓ Favorece polimorfismo.



6.4 Interface Segregation Principle (ISP)

Principio: Las clases no deben depender de interfaces que no utilizan.

Aplicación en el sistema: El patrón Observer utiliza una interfaz específica: Observer que únicamente contiene operaciones relacionadas con notificaciones. De esta manera, las clases observadoras implementan solo los métodos necesarios.

Beneficio

- ✓ Reduce dependencias innecesarias.
- ✓ Mejora claridad del diseño.
- ✓ Facilita mantenimiento.

6.5 Dependency Inversion Principle (DIP)

Principio: Los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones.

Aplicación en el sistema: El sistema utiliza interfaces y clases abstractas como:

- ✓ Observer
- ✓ IncidenteFactory
- ✓ Incidente

permitiendo que los componentes dependan de abstracciones y no de implementaciones concretas.

Beneficio

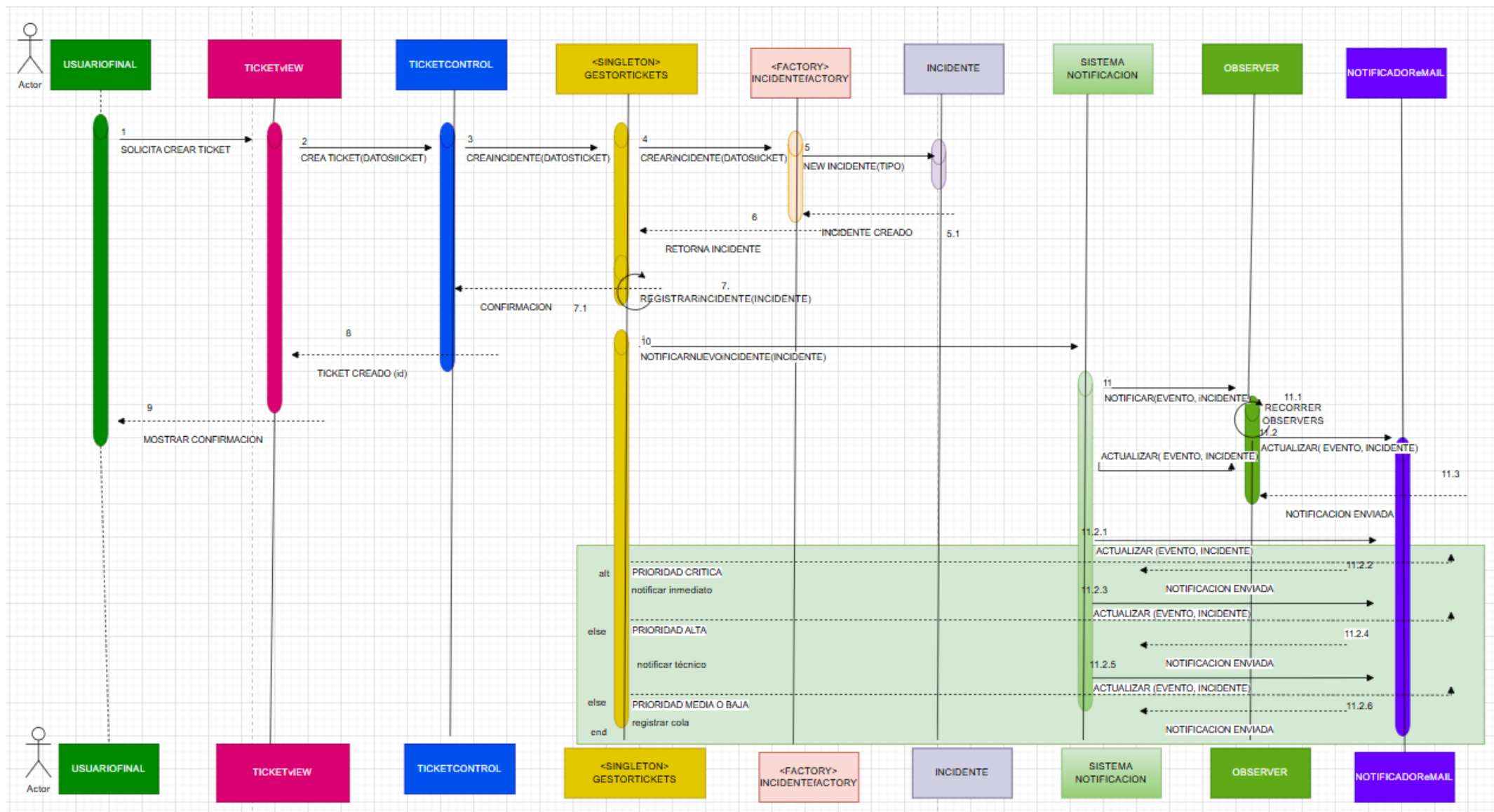
- ✓ Reduce acoplamiento.
- ✓ Mejora flexibilidad.
- ✓ Facilita reemplazo de componentes.

6.6 Importancia de SOLID en el Sistema Help Desk

La implementación de los principios SOLID permitió desarrollar una arquitectura lógica más organizada y preparada para futuras ampliaciones del sistema. Gracias a estos principios, el sistema Help Desk posee una estructura más profesional, reutilizable y adaptable a cambios tecnológicos o nuevas funcionalidades.



Diagrama de secuencia





Enlace de Diagrama de secuencias:

<https://drive.google.com/file/d/1L8IzDR6oU0jTpS2635MA4PgRzR5Bo5Tf/view?usp=sharing>

7. Evidencias de Gitflow

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git add .
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git commit -m "commit inicial: subida de la primera version de la presentacion"
[main fce9c09] commit inicial: subida de la primera version de la presentacion
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 capturas.docx
create mode 100644 ~$pturas.docx
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git commit -m "commit update: subida del archivo adicional tutorial"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git remote add origin
usage: git remote add [<options>] <name> <url>
```

Inicializa un nuevo repositorio Git local en el proyecto, permitiendo comenzar el control de versiones.

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git remote add origin https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
error: remote origin already exists.
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git branch -M main
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 440 bytes | 440.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
   e37ccb5..fce9c09  main -> main
branch 'main' set up to track 'origin/main'.
```

Conecta el repositorio local con el repositorio remoto alojado en [GitHub](https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git).



Renombra la rama principal del proyecto a main.

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
branch 'main' set up to track 'origin/main'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ checkout -b develop
bash: checkout: command not found

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git checkout -b develop
Switched to a new branch 'develop'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/develop
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ |
```

Agrega todos los archivos modificados al área de preparación (staging area) para posteriormente realizar un commit

Guarda los cambios realizados en el historial del repositorio junto con un mensaje descriptivo.

Envía los commits locales hacia el repositorio remoto en GitHub.



```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b feature/crear-ticket
Switched to a new branch 'feature/crear-ticket'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/crear-ticket)
$ git checkout -b feature/uml-diagramas
Switched to a new branch 'feature/uml-diagramas'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/uml-diagramas)
$ git add .

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/uml-diagramas)
$ git commit -m "feat: agregado diagrama UML de clases"
On branch feature/uml-diagramas
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/uml-diagramas)
$ git push -u origin feature/uml-diagramas
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/uml-diagramas' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/feature/uml-diagramas
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      feature/uml-diagramas -> feature/uml-diagramas
branch 'feature/uml-diagramas' set up to track 'origin/feature/uml-diagramas'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/uml-diagramas)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
```

Crea una nueva rama y cambia automáticamente a ella.

Fue utilizado para implementar el flujo Gitflow mediante ramas:

develop

feature/*

release/*

hotfix/*

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge feature/uml-diagramas
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b feature/secuencia-incidentes
Switched to a new branch 'feature/secuencia-incidentes'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/secuencia-i
ncidentes)
$ git add .

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/secuencia-incidentes)
$ git commit -m "feat: agregado diagrama de secuencia"
On branch feature/secuencia-incidentes
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/secuencia-incidentes)
$ git push -u origin feature/secuencia-incidentes
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/secuencia-incidentes' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/feature/secuencia-incidentes
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
```



```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/secuencia-incidentes)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge feature/secuencia-incidentes
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b feature/patrones-diseno
Switched to a new branch 'feature/patrones-diseno'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/patrones-diseno)
$ git add .
$ git commit -m "feat: implementación lógica de patrones de diseño"
On branch feature/patrones-diseno
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/patrones-diseno)
$ git push -u origin feature/patrones-diseno
total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/patrones-diseno' on GitHub by visiting:
```

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf

remote: Create a pull request for 'feature/patrones-diseno' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/feature/patrones-diseno
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      feature/patrones-diseno -> feature/patrones-diseno
branch 'feature/patrones-diseno' set up to track 'origin/feature/patrones-diseno'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/patrones-diseno)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge feature/patrones-diseno
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b feature/notificaciones
Switched to a new branch 'feature/notificaciones'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/notificaciones)
$ git add .
$ git commit -m "feat: agregado patrón observer para notificaciones"
On branch feature/notificaciones
nothing to commit, working tree clean
```

Permite cambiar de una rama a otra dentro del proyecto.

Fusiona los cambios de una rama hacia otra, integrando nuevas funcionalidades al proyecto principal.



```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/notificaciones)
$ git push -u origin feature/notificaciones
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/notificaciones' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/feature/notificaciones
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      feature/notificaciones -> feature/notificaciones
branch 'feature/notificaciones' set up to track 'origin/feature/notificaciones'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/notificaciones)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge feature/notificaciones
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b feature/documentacion
Switched to a new branch 'feature/documentacion'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/documentacion)
```

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/documentacion)
$ git commit -m "docs: agregado informe técnico y evidencias Gitflow"
On branch feature/documentacion
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/documentacion)
$ git push -u origin feature/documentacion
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/documentacion' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/feature/documentacion
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      feature/documentacion -> feature/documentacion
branch 'feature/documentacion' set up to track 'origin/feature/documentacion'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (feature/documentacion)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge feature/documentacion
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git checkout -b release/v1.0
```

Crea una etiqueta de versión para identificar versiones estables del sistema.

Envía la etiqueta creada hacia el repositorio remoto.



```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (release/v1.0)
$ git add .

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (release/v1.0)
$ git commit -m "release: versión estable 1.0"
On branch release/v1.0
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (release/v1.0)
$ git push -u origin release/v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'release/v1.0' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/release/v1.0
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new branch]      release/v1.0 -> release/v1.0
branch 'release/v1.0' set up to track 'origin/release/v1.0'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (release/v1.0)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git merge release/v1.0
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git push origin main
Everything up-to-date
```

```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git tag -a v1.0 -m "Versión final del sistema Help Desk"

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 185 bytes | 185.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
 * [new tag]         v1.0 -> v1.0

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git checkout -b hotfix/correccion-login main
Switched to a new branch 'hotfix/correccion-login'

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (hotfix/correccion-login)
$ git add .

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (hotfix/correccion-login)
$ git commit -m "fix: corrección de validación de acceso"
On branch hotfix/correccion-login
nothing to commit, working tree clean

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (hotfix/correccion-login)
$ git push -u origin hotfix/correccion-login
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'hotfix/correccion-login' on GitHub by visiting:
```



```
MINGW64:/c:/Users/jeanc/Desktop/utm/Actividad1_CruzFajardo.pdf
remote: Create a pull request for 'hotfix/correccion-login' on GitHub by visiting:
remote:   https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM/pull/new/hotfix/correccion-login
remote:
To https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM.git
* [new branch]      hotfix/correccion-login -> hotfix/correccion-login
branch 'hotfix/correccion-login' set up to track 'origin/hotfix/correccion-login'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (hotfix/correccion-login)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git merge hotfix/correccion-login
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git push origin main
Everything up-to-date

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (main)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git merge hotfix/correccion-login
Already up to date.

jeanc@jean123 MINGW64 ~/Desktop/utm/Actividad1_CruzFajardo.pdf (develop)
$ git push origin develop
```

Branch	Updated	Check status	Behind	Ahead	Pull request
main	38 minutes ago			Default	
hotfix/correccion-login	5 minutes ago		0	0	
release/v1.0	7 minutes ago		0	0	
feature/documentacion	8 minutes ago		0	0	
feature/notificaciones	10 minutes ago		0	0	
feature/patrones-diseno	12 minutes ago		0	0	
feature/secuencia-incidentes	14 minutes ago		0	0	
feature/uml-diagramas	16 minutes ago		0	0	
develop	26 minutes ago		0	0	

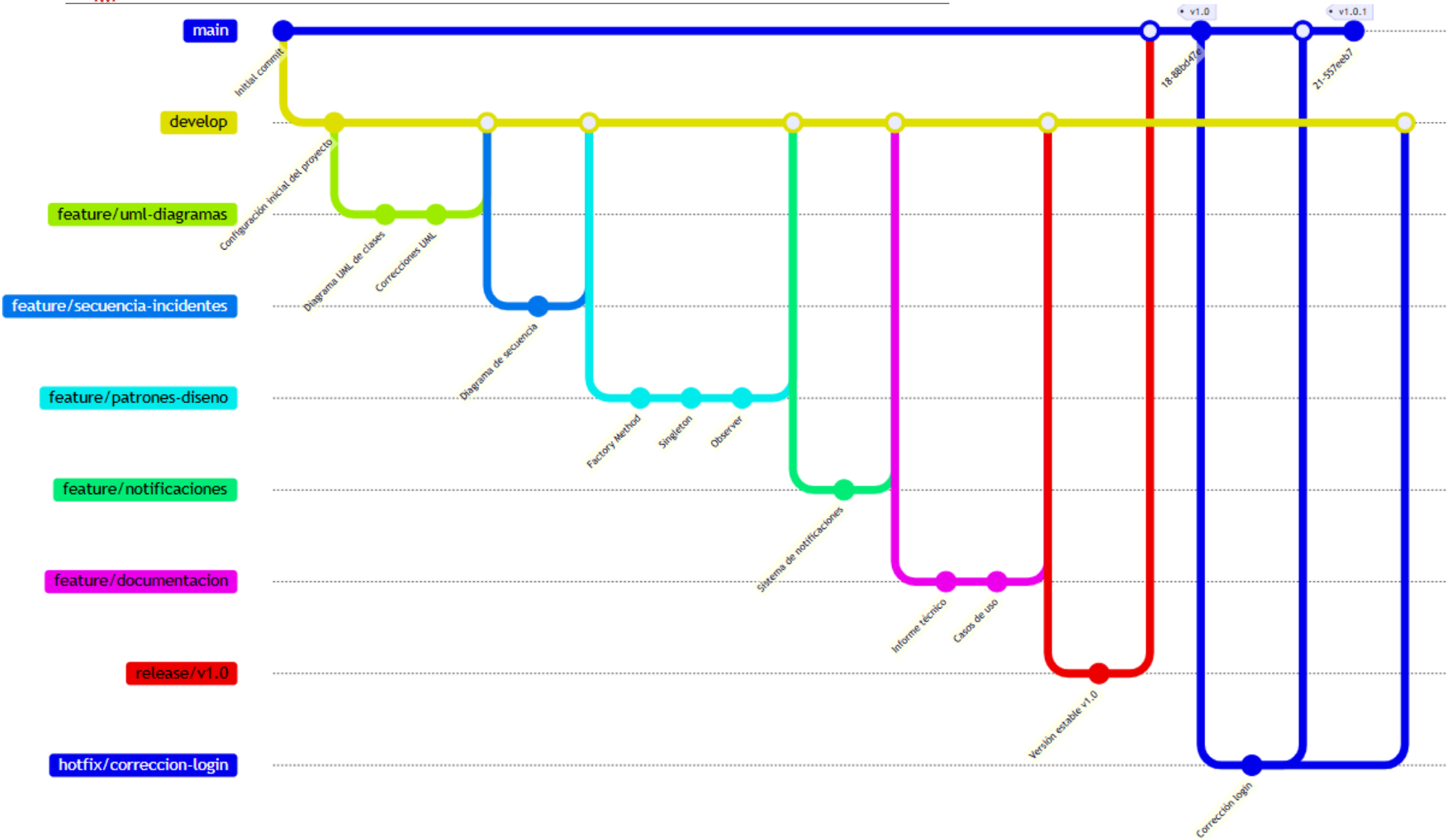
8. Flujo Gitflow Implementado

El proyecto utilizó el modelo Gitflow para organizar el desarrollo y control de versiones.

Rama	Función
main	Contiene versiones estables del sistema
develop	Rama principal de desarrollo
feature/*	Desarrollo de nuevas funcionalidades
release/*	Preparación de versiones finales
hotfix/*	Corrección rápida de errores

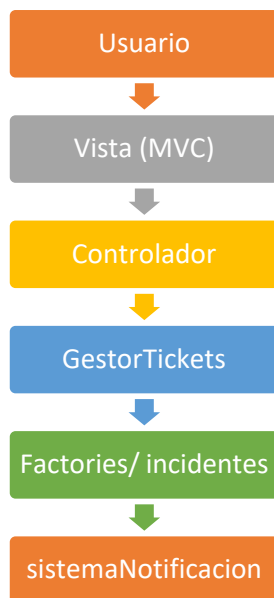
Beneficios del Uso de Gitflow

- ✓ Mejor organización del desarrollo.
- ✓ Separación de funcionalidades.
- ✓ Control de versiones profesional.
- ✓ Trazabilidad de cambios.
- ✓ Facilita mantenimiento y trabajo colaborativo.





9. Arquitectura general del sistema



Rama	Último commit	Actualizado
 main principal	Merge release/v1.0	hace 1 día
 develop	Merge feature/notificaciones	hace 2 horas
 feature/crear-ticket	feat: validaciones del formulario	hace 3 horas
 feature/notificaciones	feat: servicio de notificaciones	hace 5 horas
 feature/incidentes	feat: modulo de incidentes	hace 1 día
 release/v1.0	chore: ajustes para release v1.0	hace 1 día
 hotfix/correccion-login	fix: error al iniciar sesión	hace 30 min

ENLACE DE GITHUB

<https://github.com/jcruz2543-ctrl/Desarrollo-de-software-UTM>

ENLACE DE DIAGRAMAS

<https://drive.google.com/file/d/1L8IzDR6oU0jTpS2635MA4PgRzR5Bo5Tf/view?usp=sharing>

10. Recomendaciones

- ✓ Se recomienda implementar una base de datos relacional para almacenar de manera segura la información de usuarios, tickets e incidencias, garantizando integridad y persistencia de los datos del sistema Help Desk.
- ✓ Se recomienda integrar mecanismos de autenticación y autorización mediante control de roles, con el fin de mejorar la seguridad y restringir el acceso a funcionalidades sensibles del sistema.
- ✓ Se recomienda ampliar el sistema mediante nuevas categorías de incidentes y módulos adicionales, aprovechando la arquitectura escalable basada en patrones de diseño y principios SOLID.
- ✓ Se recomienda incorporar notificaciones en tiempo real mediante correo electrónico o aplicaciones móviles para mejorar la comunicación entre usuarios y técnicos de soporte.
- ✓ Se recomienda utilizar herramientas de monitoreo y análisis de rendimiento para identificar incidencias críticas y optimizar los tiempos de respuesta dentro del Data Center.



- ✓ Se recomienda mantener el uso del modelo Gitflow y control de versiones en GitHub para garantizar una adecuada trazabilidad de cambios y una gestión organizada del desarrollo del sistema.
- ✓ Se recomienda realizar pruebas funcionales y de rendimiento antes de la implementación del sistema en un entorno real, con el objetivo de validar el correcto funcionamiento de los módulos desarrollados.
- ✓ Se recomienda continuar aplicando diagramas UML y documentación técnica en futuras ampliaciones del sistema para facilitar el mantenimiento y comprensión de la arquitectura de software.

10. Conclusiones

1. El diseño del sistema Help Desk permitió aplicar correctamente conceptos fundamentales de ingeniería de software, tales como modelado UML, arquitectura orientada a objetos y patrones de diseño, facilitando la comprensión y organización del sistema.
2. La implementación de los patrones Singleton, Factory Method y Observer permitió construir una arquitectura escalable, desacoplada y mantenible, mejorando la reutilización de componentes y la flexibilidad del sistema ante futuros cambios.
3. El uso del modelo Gitflow mediante [GitHub](#) permitió gestionar adecuadamente las versiones y el desarrollo modular del proyecto, garantizando trazabilidad, organización y control de cambios durante el proceso de desarrollo.
4. Los diagramas UML desarrollados, incluyendo casos de uso, clases y secuencia, facilitaron la representación visual de la lógica del sistema y las interacciones entre actores, fortaleciendo el análisis y diseño de la solución propuesta.
5. La aplicación de principios SOLID y del patrón MVC contribuyó a una mejor separación de responsabilidades, permitiendo un sistema más estructurado, profesional y preparado para futuras ampliaciones.

11. Bibliografía

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
2. Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.
3. Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson Educación.
4. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The unified modeling language user guide (2nd ed.). Addison-Wesley.
5. Atlassian. (s.f.). Gitflow workflow. Atlassian Documentation. Recuperado el 16 de mayo de 2026, de Atlassian Gitflow Workflow



6. GitHub. (s.f.). GitHub documentation. Recuperado el 16 de mayo de 2026, de GitHub Documentation
7. Draw.io. (s.f.). Draw.io diagrams. Recuperado el 16 de mayo de 2026, de Draw.io
8. Lucid Software Inc. (s.f.). Lucidchart UML tool. Recuperado el 16 de mayo de 2026, de Lucidchart